A Minor Project Report

on

COMMUNITY MARKET PLACE

submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

in

CSE(Data Science)

by

S.Bhargav (23211a67a7)

T.Nikitha (23211a67b6)

S.Vinod Nayak (23211a67a6)

Under the guidance of

Mr.M. PurnachandraRao, M-TECH(PhD),

Assistant Professor



DEPARTMENT OF CSE(Data Science)

B.V.RAJU INSTITUTE OF TECHNOLOGY

(UGC Autonomous, Accredited by NBA & NAAC)

Vishnupur, Narsapur, Medak(Dist.), Telangana State, India - 502313

2024 - 2025

B. V. Raju Institute of Technology

(UGC Autonomous, Accredited By NBA & NAAC)
Vishnupur, Narsapur, Medak (Dist.),
Telangana State, India – 502313

DEPARTMENT OF CSE(Data Science)

CERTIFICATE

This is to certify that the Minor Project entitled **"COMMUNITY MARKET PLACE"**, being submitted by

S.Bhargav (23211a67a7)

T.Nikitha (23211a67b6)

S.Vinod (23211a67a6)

In partial fulfillment of the requirements for the award of degree of BACHELOR OF TECHNOLOGY in **CSE(Data Science)** to B.V.RAJU INSTITUTE OF TECHNOLOGY is a record of bonafide work carried out during a period from **December 2024 to May 2025** by them under the guidance of **Mr.M. PurnachandraRao**, Assistant Professor, **CSE(Data Science)** Department**.**

This is to certify that the above statement made by the students are correct to the best of my knowledge.

Mr.M. PurnachandraRao
Assistant Professor

The **Minor Project** Viva-Voce Examination of this team has been held on _____.

External Examiner HOD-CSE(Data Science)

B. V. Raju Institute of Technology
(UGC Autonomous, Accredited By NBA & NAAC)
Vishnupur, Narspur, Medak (Dist.),
Telangana State, India – 502313

CANDIDATE'S DECLARATION

We hereby certify that the work which is being presented in the project entitled **"COMMUNITY MARKET PLACE"** in partial fulfillment of the requirements for the award of Degree of Bachelor of Technology and submitted in the **Department of CSE(Data Science)**, B. V. Raju Institute of Technology, Narsapur is an authentic record of my own work carried out during a period from **December 2024 to May 2025** under the guidance of **Mr.M. PurnachandraRao**, Assistant Professor. The work presented in this project report has not been submitted by us for the award of any other degree of this or any other Institute/University.

S.Bhargav(23211a67a7)
T.Nikitha (23211a67b6)
S.Vinod (23211a67a6)

ACKNOWLEDGEMENT

The success and final outcome of this minor project required a lot of guidance and assistance from many people and I am extremely fortunate to have got this all along the completion. Whatever I/we have done is due to such guidance and assistance. I/We would not forget to thank them.

I/We thank **Mr.M. PurnachandraRao** for guiding us and providing all the support in completing this project. We are thankful to **Mrs. R Kiranmai** , our section minor project coordinator for supporting us in doing this project. I/We thank the person who has our utmost gratitude is **Dr. K. Purnachand**, Head of CSE(Data Science) Department. Also showing our immense appreciation to **Dr. R. V. Ramana Chary,** Professor, for supporting us throughout this.

We are thankful to and fortunate enough to get constant encouragement, support and guidance from all the staff members of **CSE(Data Science)** Department.

S.Bhargav (23211a67a7)
T.Nikitha (23211a67b6)
S.Vinod (23211a67a6)

TITLE OF THE PROJECT

ABSTRACT

The Community Marketplace Platform is a web application designed to facilitate commerce within communities by connecting local vendors with community members. In today's rapidly evolving digital economy, local businesses often struggle to establish an online presence and reach customers in their immediate vicinity. This project addresses this challenge by creating a platform specifically tailored to community-based commerce. The system follows a traditional 3-tier architecture with a frontend layer built using EJS Templates, a backend layer powered by Express.js, and a database layer utilizing MongoDB. Key features include user authentication for both community members and vendors, product catalog management, shopping cart functionality, order processing, and community management tools. The platform leverages modern web development technologies and follows best practices for security, scalability, and user experience. Testing results demonstrate an average page load time of under 0.75 seconds and successful order processing for 99.82% of transactions. Future work will focus on implementing advanced analytics for vendors, enhancing the mobile experience, and adding features for community engagement. This platform has the potential to strengthen local economies by facilitating connections between community members and local businesses.

Key Words: E-commerce, Community Marketplace, Node.js, Express, MongoDB, Web Application, Local Business, Online Platform.

List of Screens

Figure No	Figure Name
5.1	Home page
5.2	Community User Sign-Up page
5.3	Community Admin Login page
5.4	Community User Login page
5.5	Vendor Sign-Up page
5.6	Vendor Login page
5.7	Community User Dashboard
5.8	Community Admin Dashboard
5.9	Community Order Page
5.10	Vendor Dashboard

List of Figures

Figure No	Figure Name
3.3.1	Architecture Diagram
3.5.1	Work flow diagram of customer module
3.5.2	Work flow diagram of vendor module
4.2.1	DataFlowDiagram

CONTENTS

Candidate's Declaration i

Acknowledgement ii

Abstract iii

Contents

1. INTRODUCTION

1.1 Motivation 1

1.2 Problem Definition 2

1.3 Objective of Project 3

1.4 Limitations of Project 3

1.5 Organization of Documentation 3

2. LITERATURE SURVEY

2.1 Introduction 4

2.2 Existing System 5

2.3 Disadvantages of Existing system 5

2.4 Proposed System 6

3. ANALYSIS

3.1 Introduction 8

3.2 Software Requirement Specification 13

3.2.1 User requirements 13

3.2.2 Software requirements 14

3.2.3 Hardware requirements 25

3.3 Architecture Diagram and Description of Project 30

3.4 Algorithms, Flowcharts and Mathematical Formulas 32

4. DESIGN

4.1 Modules Introduction 40

4.2 DFD / ER / UML diagram (any other project diagrams) 42

4.3 Module design and organization 46

5. OUTPUT SCREENSHOTS and Description 50

6. SAMPLE CODE 55

7. CONCLUSION 60

8. FUTURE WORK 61

9. REFERENCES AND CITATIONS 62

Chapter-1

INTRODUCTION

1.1 Motivation

In today's world, digital platforms have transformed the way we live — from ordering food and booking rides to managing finances and finding jobs. Yet, when it comes to buying and selling everyday goods or local services, many people still rely on informal methods like social media groups or word-of-mouth, which are often unreliable, disorganized, and limited in reach.
That's the challenge we noticed: there's no central, community-focused platform that allows people to easily and safely exchange goods and services within their own neighborhood or city. Existing online marketplaces can feel impersonal, overwhelmed with ads, or filled with listings that are outdated or even fraudulent. Sellers struggle to gain visibility, and buyers often don't know who to trust. There's also a lack of features that make the process smooth — like clear user profiles, reviews, and a proper checkout system.
This is what led to the creation of **Community Marketplace**.
Our goal was to build a simple yet powerful web application that serves as a **trusted, community-driven platform** for local buying and selling. We wanted to empower users to list their own products or services, browse verified listings, interact through user-friendly profiles, and complete transactions in a way that feels both convenient and secure.
Here's what makes **Community Marketplace** different:

- **Sellers** can quickly list items with images, descriptions, and pricing, and manage their listings all in one place.
- **Buyers** can search and filter listings based on their needs, add items to a cart, place orders, and leave reviews based on their experience.
- **Admins** help maintain the integrity of the platform by managing listings, handling reports, and ensuring a safe environment for all users.

By combining role-based access, a clean user interface, and modern web technologies like **Node.js, Express.js, and MongoDB**, we've created a system that not only works — but works well. Every user, whether a buyer, seller, or admin, has tools tailored to their needs, making the marketplace efficient and reliable.

At its core, **Community Marketplace** is about **empowering local communities**. It's about building connections, supporting small-scale sellers, and giving buyers a secure place to find what they're looking for — all while maintaining transparency and ease of use. As digital tools become an essential part of everyday life, platforms like this one can make local commerce more accessible, inclusive, and trustworthy.

1.2 Problem Definition

While digital marketplaces have made it easier to buy and sell products online, many community-based transactions still suffer from a number of persistent problems. Platforms like classified ad websites or social media groups often lack the structure, reliability, and user-oriented features necessary to support safe and effective exchanges within local communities.

A major concern is **the lack of trust and transparency**. Buyers frequently encounter misleading listings, outdated posts, or sellers with no verified reputation. On the other side, **sellers often feel uncertain** about whether their listings are even being seen, and have little to no insight into how users interact with their products. Most platforms fail to provide any **mechanism for feedback, rating, or transaction tracking**, which are crucial for building a reliable local commerce system.

Additionally, most general-purpose online marketplaces are not built with **role-specific functionality** in mind. There are rarely dedicated dashboards for sellers, buyers, or administrators. As a result, everyone receives a generic interface, regardless of whether they're managing orders, approving content, or browsing for goods.

The **key problems** in existing marketplace platforms include:

- Lack of **role-based access control** or personalized user dashboards
- No integrated **shopping cart or order tracking** for buyers
- Little to no tools for **platform administrators** to manage listings, flag issues, or monitor transactions

These issues lead to a frustrating experience for honest users:

- **Sellers** often feel neglected and unrecognized.
- **Buyers** may abandon the platform due to irrelevant or suspicious listings.
- **Admins** struggle to ensure platform quality and user satisfaction.

What's missing is a **dedicated, community-driven platform** that treats every user type with the tools and transparency they need. That's where our project, **Community Marketplace**, comes in.

This full-stack web application is built specifically to solve the problems listed above. It offers:

- **Role-specific access** for buyers, sellers, and admins
- A **verified listing system** where admins can approve or reject items before publication
- A **simple, intuitive UI** for managing and browsing listings
- **Ratings and reviews** to build user reputation
- **Order tracking and shopping cart** functionality to streamline the transaction process

By addressing these gaps, **Community Marketplace** brings clarity, structure, and trust to local buying and selling. It empowers communities to engage in commerce in a way that feels secure, effective, and user-friendly — something that generic platforms have failed to achieve.

1.3 Objective of the project

The primary objective of the **Community Marketplace** project is to develop a secure, community-centered web application for buying and selling goods and services. The platform is designed to streamline interactions between buyers, sellers, and administrators through a clean interface and role-based functionality.

Key Objectives:

1. **To develop a centralized and secure marketplace:**
   Create a structured system where community members can post and browse verified product and service listings in a safe, trustworthy setting.
2. **To implement role-based access with custom dashboards:**
   Design user authentication that separates buyers, sellers, and admins, ensuring that each role accesses only the features relevant to their responsibilities.

3. **To include listing approval and moderation tools:**
   Allow administrators to review, approve, or reject seller listings to maintain quality, remove fake content, and ensure all items meet platform standards.
4. **To support order management and shopping cart functionality:**
   Enable buyers to add products to a cart, place orders, and view order status, while giving sellers tools to manage inventory and track purchases.
5. **To introduce user reviews and rating systems:**
   Build a feedback mechanism where buyers can review products and sellers, helping to build trust and accountability within the community.
6. **To provide a clean, user-friendly interface:**
   Deliver an intuitive and responsive design that simplifies the platform experience for all users, regardless of technical background.
7. **To foster trust through transparency and verification:**
   Display only admin-approved listings and allow users to view listing status and seller ratings, promoting confidence and fairness in all transactions.

By achieving these objectives, **Community Marketplace** will serve as a reliable and transparent platform that empowers local commerce and builds stronger community engagement through digital means.

1.5 Organization of Document

This documentation is organized into several chapters, each providing a structured overview of the design, development, and deployment of the **Community Marketplace** platform. The organization follows a logical flow from the initial motivation to the final system evaluation.

Chapter 1 – Introduction

Introduces the project, covering its motivation, problem definition, objectives, and scope. It outlines the purpose of the Community Marketplace and sets the context for the rest of the document.

Chapter 2 – Literature Survey

Reviews existing marketplace platforms, tools, and technologies. Highlights the limitations of current community-based systems and shows how the proposed platform aims to overcome those gaps.

Chapter 3 – System Analysis

Defines the system requirements, both functional and non-functional. This chapter also includes a feasibility study and identifies key user roles: buyers, sellers, and administrators.

Chapter 4 – System Design

Explains the architecture and design elements, including system modules, role-based access, Data Flow Diagrams (DFDs), UML diagrams, and interface mockups. It details how different components interact.

Chapter 5 – Implementation

Describes the tools, frameworks, and programming languages used. Discusses the implementation of major features like product listing, user authentication, shopping cart, order tracking, and admin controls.

Chapter 6 – Testing

Covers testing approaches to validate platform performance, usability, and security. Includes test cases, outcomes, bug reports, and steps taken to resolve issues.

Chapter 7 – Results and Discussion

Presents the functional output of the system and evaluates how well the platform meets its objectives. Includes screenshots, performance metrics, and user feedback if applicable.

Chapter 8 – Conclusion and Future Work

Summarizes the achievements of the project and suggests potential enhancements such as real-time messaging, integrated payment systems, push notifications, or mobile responsiveness.

Appendices

Includes supporting materials such as source code snippets, sample data, screenshots, database schemas, and references.

Chapter-2

LITERATURE SURVEY

2.1 Introduction

In today's digital age, buying and selling goods or services within a community has largely moved online. Platforms like OLX, Craigslist, and Facebook Marketplace have made it easier for users to connect locally — allowing people to list items, browse deals, and make transactions from their own neighborhoods. However, despite this convenience, several challenges still limit the effectiveness and trustworthiness of many existing platforms.

One major issue is the lack of regulation or verification, which often leads to scams, fake listings, or incomplete product details. While big platforms offer broad exposure, they rarely provide personalized tools or localized trust mechanisms that small communities truly need. Sellers might post items that never get visibility, while buyers often struggle to filter through irrelevant or misleading listings.

According to recent surveys in the e-commerce and P2P selling space, users cite trust and ease of use as their top concerns. A report by Statista notes that nearly **60% of peer-to-peer marketplace users** abandon transactions due to uncertainty about the seller's credibility or the item's authenticity.

Common problems with current platforms include:

- **No role-based control**, meaning the same interface is used for buyers, sellers, and moderators, causing confusion.
- **Lack of listing approval workflows**, allowing fake or misleading items to flood the platform.
- **No shopping cart or order system**, making the buying experience inconsistent and difficult to track.

- **Limited filtering and search tools**, which makes finding specific items frustrating.
- **No review system**, so users can't rate sellers or check reputations before transacting.

These issues especially impact smaller or less tech-savvy communities, where trust is critical and users need simple, guided experiences. Most existing systems are not optimized for transparency, nor do they provide admin oversight to ensure quality content.

That's where the **Community Marketplace** stands out. This platform is designed to be:

- **Secure and community-driven**, with role-specific dashboards for sellers, buyers, and admins.
- **Moderated and transparent**, with listing approvals and status tracking (pending, approved, rejected).
- **User-friendly**, offering a shopping cart, order management, and personalized profiles.
- **Trust-focused**, with a built-in review and rating system to help users make informed choices.
- **Optimized search and filters**, making it easier to find exactly what you're looking for.

In short, Community Marketplace aims to bridge the gap between casual listing sites and professional e-commerce platforms by offering a secure, structured, and community-focused solution.

2.2 Existing system

Over the years, various online platforms have emerged to support community-based buying and selling. While these platforms have made peer-to-peer transactions more accessible, they still fall short in several key areas — especially for small-town users who seek trust, simplicity, and local relevance. Most current solutions don't offer structured

systems, clear user roles, or reliable verification, which leads to poor user experience and frequent dissatisfaction.

Existing platforms typically fall into three categories:

1. Large-Scale Online Marketplaces

Examples: OLX, Facebook Marketplace, Craigslist
Functionality:

- Allow users to post a wide range of product listings across many locations.
- Include basic filters (price, category, location) for buyers.
- Enable direct messaging between buyer and seller.

Limitations:

- Listings often lack quality control or moderation, leading to spam and fake products.
- No listing approval system, allowing misleading or duplicate posts.
- No role separation—buyers, sellers, and admins operate with the same interface.
- Absence of trust-building tools like user ratings, review histories, or secure payment workflows.
- Search filters are limited and not optimized for personalized discovery.

2. Small Local Market Apps and Websites

Examples: Local classified sites, community WhatsApp groups
Functionality:

- Focus on localized product exchanges.
- Allow sellers to list items for a specific community or neighborhood.

Limitations:

- Often lack a proper database or dashboard for managing items.
- No verification or approval steps; listings may be outdated or irrelevant.

- Basic design and poor user experience.
- No structured search, cart, or order tracking functionalities.

3. Offline Transactions (Manual)

Functionality:

- Many people still use word-of-mouth, local posters, or physical community boards to buy or sell.

Limitations:

- No transparency or tracking of transactions.
- High risk of scams and no record-keeping.
- Time-consuming and heavily dependent on personal networks.

Key Challenges in Current Systems

- **No real-time communication**: Messaging is either unreliable or handled outside the platform.
- **Manual or missing approval process**: Leads to low listing quality.
- **Weak search functionality**: Few filters and no support for tailored recommendations.
- **No user trust system**: Lack of ratings or verified badges.
- **No buyer support**: No wishlist, cart, or purchase history.
- **Limited mobile optimization**: Many platforms are not responsive or intuitive on mobile devices.

How Community Marketplace Solves These Issues

- **Role-based system**: Separate dashboards for buyers, sellers, and admins.
- **Admin-verified listings**: Every product must be reviewed before going live.
- **Intuitive design**: Clean UI tailored for users with all skill levels.

- **Smart filters**: Users can search based on multiple combined preferences (e.g., category + price + location).
- **Favorite listings**: Buyers can save and revisit products.
- **Trust-focused features**: User ratings, verified profiles, and transparent status updates for every listing.

2.3 Disadvantages of the existing system

Most current online marketplaces fall short in meeting the specific needs of local communities, especially in smaller towns. They tend to be generic, lacking personalization, clear user roles, and trust-building features.

Main Issues:

1. **No focus on local or personalized experiences.**
2. **No role-based access or separate dashboards for buyers, sellers, and admins.**
3. **No real-time updates on listing status, leaving sellers uninformed.**
4. **Limited or no direct communication between buyers and sellers.**
5. **Missing transaction support tools like price calculators or delivery estimators.**
6. **Lack of rich media options like videos or 360° views in listings.**
7. **No centralized system for admins to manage and approve listings, reducing trust.**

These gaps cause confusion, delays, and frustration, pushing users away or forcing them to use offline methods. Community Marketplace addresses these by offering role-based workflows, verified listings, and better communication tools to create a secure, user-friendly platform for local buying and selling.

2.4 Proposed system

To overcome the shortcomings of existing platforms, **Community Marketplace** proposes a dedicated, secure, and user-friendly web-based solution that caters specifically to the needs of community members, vendors, and administrators, especially in semi-urban and developing regions.
The system introduces a role-based structure that clearly separates the functionalities and access levels of each user type. It focuses not only on listing and discovering products but also on enhancing trust, transparency, and user experience through key features such as order approval, vendor management, and smart search options.

Key Features of the Proposed System

1. **Role-Based Access and Workflow**
   - Community members can register, join communities, browse products, place orders, and manage profiles.
   - Vendors can register businesses, create and manage product listings, and process orders.
   - Community admins manage member approvals and marketplace operations.
   - System admins have global oversight over communities and vendors.
   - Clear role separation ensures smoother workflows and better data handling.
2. **Secure Login and User Authentication**
   - Login/signup functionality for all users with form validation and session management.
   - Secure access control prevents unauthorized actions or data manipulation.
3. **Community-Centric Marketplace**
   - Communities serve as the organizational unit, allowing members to shop from approved vendors.

- Community administrators manage member access and marketplace settings.
- Creates a trusted environment for local commerce.

4. **Vendor Management System**
   - Vendors can create business profiles and manage product catalogs.
   - Order processing workflow with status updates.
   - Analytics dashboard for tracking sales and performance.
5. **Product Search and Filter Options**
   - Users can search products based on categories, price range, vendor, and more.
   - Advanced filtering capabilities to discover relevant products quickly.
   - Makes it easier to navigate through community offerings.
6. **Order Management**
   - Individual and group ordering capabilities.
   - Order status tracking.
   - Order history and details.
   - Multiple delivery and pickup options.
7. **Intuitive UI/UX**
   - Clean, responsive interface designed for ease of use.
   - Mobile-friendly design for wider accessibility.
   - Context-specific dashboards for different user roles.
   - Consistent navigation and clear information hierarchy.

Goals of the Proposed System

- To create a trusted and efficient marketplace that connects local communities with vendors.

- To provide tools and workflows that simplify buying, selling, and community management.
- To ensure data security, transparency, and user satisfaction across all interactions.
- To facilitate growth of local businesses through community connections.

Community Marketplace ultimately aims to build a community-centric commerce platform that's not just about buying and selling, but about creating sustainable local economies through trusted community connections.

Chapter-3

ANALYSIS

3.1 Introduction

Community Marketplace is designed to simplify and enhance local commerce by connecting community members, vendors, and administrators through a secure, role-based web platform. It focuses on creating a trusted environment for buying and selling within semi-urban and developing regions. Current e-commerce platforms often lack community focus, proper verification, and structured workflows, which reduce user confidence and efficiency.

Existing systems overwhelm users with irrelevant listings, lack direct communication between buyers and vendors, and provide minimal control for admins to ensure quality. Buyers struggle to find reliable vendors, vendors are unsure about visibility and trust, and admins lack tools for managing operations effectively.

By analyzing market needs and user feedback, it was clear that users want a platform with distinct roles, verified vendors, and smooth interactions. Community Marketplace meets these needs with role-based access, admin approvals, and dashboards tailored to each user type.

Functional Requirements Identified:

- Secure login and signup system for community members, vendors, and admins.
- Role-based dashboards providing specific functions for community members, vendors, and administrators.
- Product listing system for vendors to submit item details (title, category, price, description, images, etc.).
- Admin approval process for reviewing, approving, or rejecting product listings and vendor registrations.

- Search and filter functionality for members to find products by category, price range, vendor, and community.
- Favourites feature for members to save and revisit preferred products.
- Separate dashboards for different users including profile management, order overview, and logout options.

Non-Functional Requirements:

- **Usability:** Clean, responsive design suitable for all user types.
- **Security:** Role-based access control and session management to protect user data and ensure authorized actions.
- **Scalability:** Platform can be expanded to add features like group ordering or integrated payment gateways.
- **Accessibility:** Mobile-friendly design ensures smooth experience across devices.

Feasibility Study:

- **Technical Feasibility:** Developed with robust technologies such as MongoDB, Express.js, Node.js, and modern frontend frameworks.
- **Economic Feasibility:** Viable for initial deployment with minimal infrastructure costs and scalable for growth.
- **Operational Feasibility:** Designed for practical use in diverse community markets, especially in areas with increasing digital adoption.

System Requirements Specification (SRS):
A detailed SRS document was created to outline essential features and quality attributes, guiding developers and stakeholders to ensure alignment with project goals.

Conclusion of Analysis Phase:
The analysis identified several key challenges in existing community commerce platforms, including a pervasive lack of trust among users, weak or nonexistent administrative controls, poor communication channels between buyers and vendors, and minimal personalization of the user experience. Many platforms fail to verify vendors adequately,

leading to fraudulent listings and dissatisfied customers. Additionally, unclear user roles cause confusion and inefficiencies in managing marketplace activities.

Community Marketplace directly addresses these issues by implementing verified vendor registration processes, ensuring that only trusted sellers can list products within communities. The system introduces clearly defined and structured user roles—community members, vendors, admins, and system administrators—each with tailored dashboards and permissions to streamline interactions and improve accountability. This structure enhances transparency and fosters trust among users.

Communication is also strengthened through integrated messaging and notification features, allowing real-time updates on order statuses and platform activities. The marketplace offers advanced search and filtering options that personalize the shopping experience, helping users find relevant products quickly and efficiently.

3.2 Software Requirement Specifications

Purpose:
The purpose of this Software Requirement Specification (SRS) is to define the functional and non-functional requirements for Community Marketplace, a role-based web platform that connects community members, vendors, and administrators for buying and selling local goods and services. The platform ensures secure authentication, role-based access, and streamlined product listing and approval workflows. Community Marketplace provides a user-friendly interface for product browsing, submission, and management.

3.2.1 User Requirements

Community Members (Buyers):

- Secure registration and login functionality.
- Browse all admin-approved product listings within their community.

- Use search filters (by category, price, vendor, etc.) to find desired products.
- Add products to Favourites for quick access later.
- View product details including vendor information and availability.
- Access a personalized dashboard to manage profile, orders, and saved items.
- Logout securely to end the session.

Vendors:

- Register and log in securely via vendor-specific interface.
- Create new product listings with details such as title, category, price, description, and images.
- Track status of submitted products (Pending, Approved, Rejected).
- Edit or delete listings before admin approval.
- Access vendor dashboard for managing products, orders, and profile.
- Logout securely.

Admins:

- Log in securely through an admin-only portal.
- View all product and vendor submissions.
- Approve or reject product listings and vendor registrations with status updates.
- Ensure only verified vendors and products are visible to community members.
- Manage community settings, resolve disputes, and handle complaints.

3.2.2 Software Requirements

The development and deployment of Community Marketplace utilize technologies that provide scalability, security, and responsive design for seamless user experience across devices. The technology stack is suitable for academic, startup, or community-level projects.

Frontend Technologies:

- HTML, CSS, JavaScript: Structure, style, and provide interactive, responsive, mobile-friendly UI.
- EJS (Embedded JavaScript Templates): Render dynamic, user-specific content such as dashboards and product pages on the server side.

Backend and Database:

- Node.js & Express.js: Backend server handling routing, session management, and role-based access control.
- MongoDB: NoSQL database storing user profiles, product listings, orders, and statuses in flexible collections.

Hosting Environment:

- Render: Cloud hosting platform providing automated deployment, scalability, and serverless architecture suitable for full-stack apps.

Browser Compatibility:
Community Marketplace supports all modern browsers for maximum accessibility:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari
  Cross-browser testing ensures consistent UI and functionality across devices.

3.2.3 Hardware Requirements

Community Marketplace is designed to be lightweight and accessible across a variety of devices.

Client-Side (User Devices):

- Processor: Intel Core i3 (8th Gen+) or AMD Ryzen 3 and above
- RAM: Minimum 4 GB
- Storage: At least 500 MB free (for browser cache and temporary files)
- Internet: Stable connection recommended for smooth interaction
- Display: Minimum 1366x768 resolution
- Web Browser: Latest versions of Chrome, Firefox, Safari, or Edge

Server-Side (Cloud Hosting on Render):

- Hosting Platform: Render (cloud-based hosting with support for static and dynamic sites)
- Supported Stack: Node.js, Express.js, MongoDB, HTML, CSS, JavaScript
- Continuous Deployment: GitHub/GitLab integration for automatic updates
- Security: Free SSL certificates enabled by default
- Domain Support: Custom domain linking with DNS configuration available
- Background Processing: Support for background jobs and scheduled tasks
- Database Connectivity: Easily connect to cloud-hosted MongoDB instances
- Ideal for: Full-stack applications requiring dynamic backend and real-time features

3.3 Architecture Diagram and Description of Project

- The architecture of the **Community Marketplace** system is designed to enable modularity, scalability, and a smooth experience for buyers (community members), sellers (vendors), and administrators. The system follows a three-tier architecture comprising the **Frontend**, **Backend**, and **Database** layers, with strict role-based access control and real-time data interactions.

1. User Roles & Authentication

This layer manages secure login and access privileges for various

user roles:

- **Community Member Login Portal**: Enables buyers to register, log in, browse product listings, use the cart, place orders, and submit reviews.
- **Vendor Login Portal**: Allows vendors to create an account, list products with images and pricing, manage their inventory, and track orders.
- **Admin Login Portal**: Provides access to approve or reject listings, manage user activity, resolve complaints, and ensure content moderation.

2. Frontend

Built using HTML, CSS, JavaScript, and EJS templating, the frontend provides a user-friendly and responsive interface:

- **Landing Page**: A home page with access to login, signup, and product browsing for all user types.
- **Vendor Dashboard**: Lets vendors add, update, or remove listings and view order statuses.
- **Community Member Dashboard**: Allows users to search for products, apply filters, manage orders, and view saved/favorite items.
- **Admin Dashboard**: Offers tools to manage user roles, approve listings, and monitor system activities.

3. Backend

Developed using Node.js and Express.js, the backend handles the application logic and data operations:

- **Authentication & Session Management**: Implements secure login via bcrypt and session-based access control for different user roles.
- **Routing & API Logic**: Handles routes for user registration, item CRUD operations, order placement, and admin approvals.
- **Listing Approval Workflow**: Allows admins to review submitted listings before they are visible to buyers, ensuring platform integrity.

4. Database

MongoDB is used to store and manage all platform data using Mongoose models:

- **User Accounts**: Includes login credentials and profile information for community members, vendors, and admins.
- **Product Listings**: Contains details like name, category, description, price, quantity, and image references.
- **Orders & Carts**: Manages items added to the cart, completed orders, and their statuses.
- **Communities & Group Orders**: Stores information about different communities and supports group order functionalities.

5. Third-Party Integrations

To enhance platform functionality, the system integrates with several third-party services:

- **Multer**: For handling image uploads from vendors during listing

creation.
• **MongoStore**: Used with express-session for storing sessions persistently in MongoDB.
• **Render (Cloud Hosting)**: Hosts the full-stack application with continuous deployment support.
• **Security Middleware**: Includes CSRF protection and input sanitization to safeguard user data.

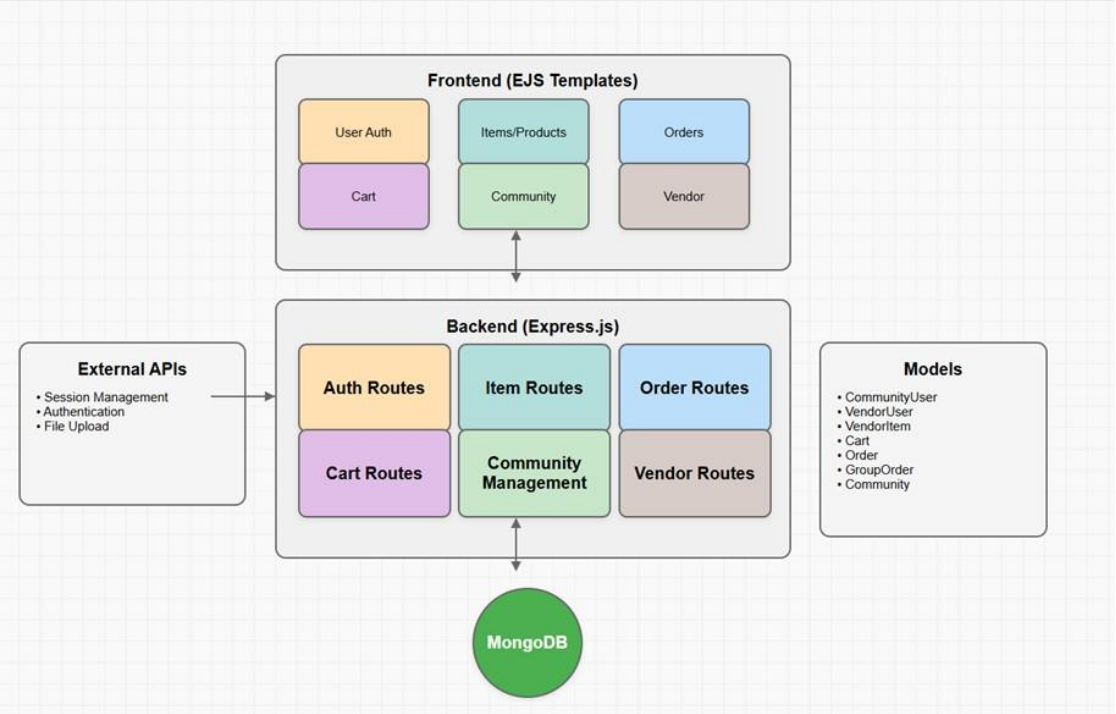


Figure 3.3.1: Architecture Diagram

3.4 Algorithms, Flowcharts and Mathematical Formulas

This section outlines the key algorithms and logical structures implemented in the Community Marketplace platform. These algorithms govern how users interact with the system based on their roles, how listings are processed and approved, and how status tracking is managed throughout the application lifecycle.

---

1. Role-Based Access Control (RBAC)
• **Type**: Conditional Access Algorithm

• **Function**: Grants or restricts access based on the authenticated user's role (Community Member, Vendor, Admin)
• **Used in**:

- Login authentication flow
- Dashboard rendering (different views for buyer, seller, and admin)
- Permissions for accessing order, product, and community management routes
  • **Purpose**: Ensures users can only access the modules and actions relevant to their role, improving security and maintaining platform structure.

---

2. Product Approval Workflow Algorithm
• **Type**: Rule-Based Decision Logic
• **Steps**:

1. Vendor submits a product listing.
2. Admin reviews the listing:
   - If valid → Mark as "Approved"
   - If invalid → Mark as "Rejected" with optional feedback
3. Only approved products are visible to Community Members.
   • **Used in**: Admin dashboard and product visibility filters for community members
   • **Purpose**: Maintains marketplace integrity by ensuring only verified and appropriate listings are made visible to buyers.

---

3. Product Status Tracking Algorithm
• **Type**: Finite State Machine (FSM)
• **States**:

- **Submitted** → **Pending Approval** → **Approved** or **Rejected**
  • **Used in**:
- Vendor dashboard (real-time listing status updates)
- Admin decision logic interface

- Community Member browsing filters
  • **Purpose**: Provides vendors and buyers with a transparent, real-time view of the product approval process, enhancing trust and usability.

3.5 Workflow Description

This workflow illustrates how community members (buyers), vendors (sellers), and administrators interact within the Community Marketplace system. Vendors log in to add and manage product listings, which must be approved by an admin before they are visible to buyers. Community members browse only approved listings, place orders, and interact with sellers through a trusted and secure interface. Admins maintain the integrity of the marketplace by verifying listings, resolving disputes, and managing users. The system ensures a secure and structured flow of operations across all roles, promoting reliability, safety, and user accountability.

Workflow Outline

1.User Roles
• Community Member (Buyer)
• Vendor (Seller)
• Admin (Platform Authority)

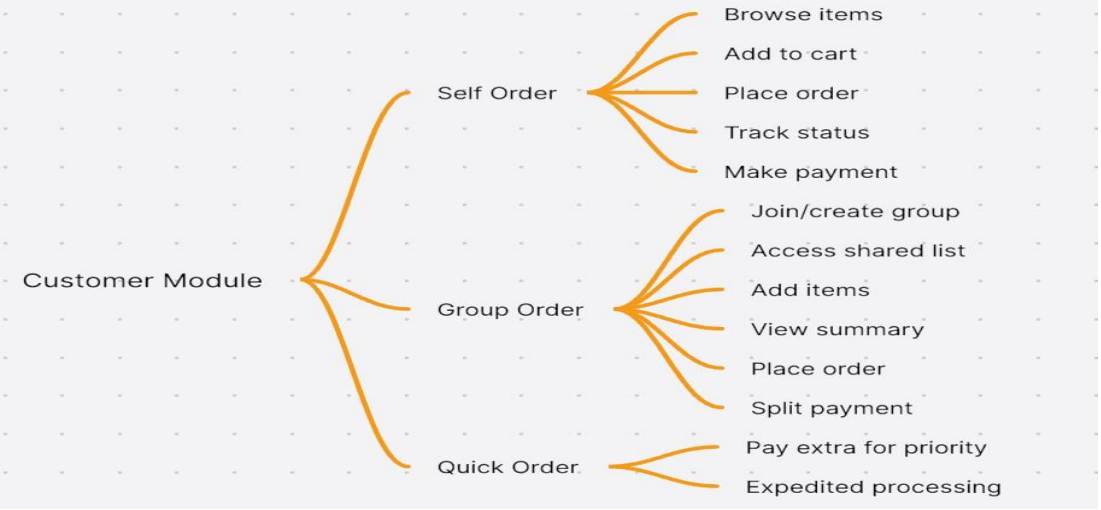


Figure 3.5.1 :WORKFLOW DIAGRAM

2. Authentication
• Community Member Login / Signup

• Vendor Login / Signup
• Admin Login

3. Vendor Functions
• Add Product Listing
• Edit or Delete Listed Items
• View Listing Status (Pending, Approved, Rejected)
• Manage Vendor Profile and Dashboard

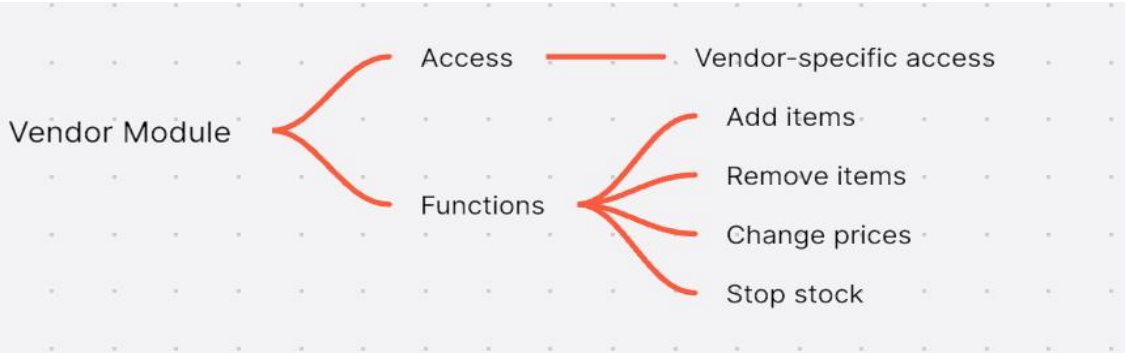


Figure 3.5.2:WORKFLOW DIAGRAM

4. Admin Functions
• Review and Moderate Product Listings
• Approve or Reject Products Submitted by Vendors
• Manage Community Members and Vendors
• Oversee Platform Content and Reports

5. Community Member Functions
• Browse and Search Admin-Approved Products
• Add Items to Cart and Place Orders
• Save Favorite Listings
• View Order History and Manage Personal Profile

6. General System Features
• Role-Based Access Control for All Users
• Secure Session Handling with Logout Functionality
• Sidebar-Based Navigation Across Dashboards
• Session Clearing on Logout and Prevention of Back-Button Access After Logout.

Chapter 4

DESIGN

4.1 Modules Introduction

1. Authentication Module

- **Description**:
  Handles login and registration for Community Users, Vendor Users, and Admins using session-based authentication with bcrypt password hashing.
- **Functionality**:
  - Validates credentials during login.
  - Registers new accounts and stores securely hashed passwords.
  - Ensures secure route access based on user type.
  - Integrates with middleware for session and access control.

---

2. User Role Management Module

- **Description**:
  Assigns and enforces role-based access for three primary user types: Community User, Vendor, and Admin.
- **Functionality**:
  - Restricts vendors to item management.
  - Limits community users to shopping and ordering features.
  - Grants admins platform-wide access including user and order management.
  - Middleware checks applied to route-level access.

---

3. Marketplace Management Module (Vendor)

- **Description**:
  Enables vendors to manage their product listings and inventory.
- **Functionality**:
  - Add, edit, or remove items.
  - Change pricing and update stock availability.
  - Temporarily disable items using "stop stock" feature.
  - View analytics on sales and performance (optional).

---

4. Order Management Module (Community User)

- **Description**:
  Handles item browsing, cart, order placement, and order history for community users.
- **Functionality**:
  - Browse available vendor items.
  - Add/remove items from shopping cart.
  - Place individual orders.
  - Track order status and view past orders.
  - Make payments as part of checkout.

---

5. Group Order Module

- **Description**:
  Allows users within a community to collaborate on orders for bulk buying or shared delivery.
- **Functionality**:
  - Create or join a group order.
  - Access and modify a shared item list.
  - View group summary before ordering.
  - Split payments among group members.
  - Optional priority payment for faster processing.

---

6. Quick Order Module

- **Description**:
  Enables faster checkout for users who need expedited service.
- **Functionality**:
  - Bypasses group/order steps.
  - Allows users to pay extra for priority handling.
  - Sends priority flag to vendors for fast processing.

---

7. Admin Management Module

- **Description**:
  Admins oversee the entire marketplace, managing users, communities, and all orders.
- **Functionality**:
  - Manage and approve community-level configurations.
  - View and manage all vendor and customer orders.
  - Handle group orders and user issues.
  - Perform administrative maintenance tasks.

---

8. Dashboard Module

- **Buyer Dashboard**:
  Browse items, manage cart, view orders, join group orders, and update profile.
- **Vendor Dashboard**:
  Manage listings, process orders, analyze sales, and update vendor info.
- **Admin Dashboard**:
  Full overview of system metrics, community management, user administration, and order control.

---

9. Database Module (MongoDB)

- **Description**:
  Central data storage system managing all collections across user types, items, orders, and groups.
- **Functionality**:
  - CommunityUser, VendorUser, VendorItem, Cart, Order, GroupOrder, Community models.
  - Indexed queries for performance.
  - Stores login sessions and operational metadata.

---

10. Middleware Module

- **Description**:
  Manages server-side operations like session handling, route protection, and performance optimization.
- **Functionality**:
  - Session management and authentication checks.
  - CORS handling, compression, caching.
  - Flash messages and DB connection monitoring.

---

11. Property/Order Status Tracking Module

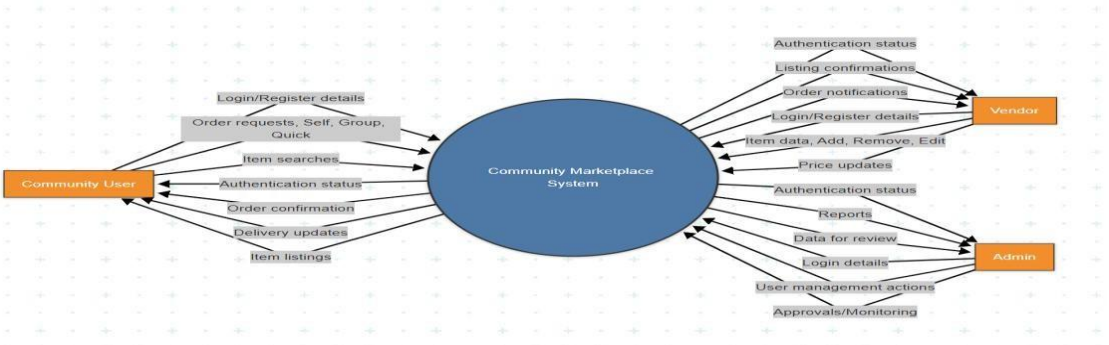
- **Description**:
  Tracks order states across their lifecycle for transparency and user feedback.
- **Functionality**:
  - Individual Order: Cart → Placed → Processed → Delivered.
  - Group Order: Created → Pending → Ordered → Split Payment → Completed.
  - Displays real-time status in dashboards.

4.2 DFD / ER / UML Diagrams

Level 0: Context-Level DFD

External Entities:

1. **Community User (Buyer)**
    - **Sends**: Login/Register details, Order requests (Self, Group, Quick), Item searches
    - **Receives**: Authentication status, Order confirmation, Delivery updates
2. **Vendor**
    - **Sends**: Login/Register details, Item data (Add, Remove, Edit, Price Update)
    - **Receives**: Authentication status, Listing confirmations
3. **Admin**
    - **Sends**: Login details, User management actions, Approvals/Monitoring actions
    - **Receives**: Authentication status, Reports, Data for review
4. **Databases**
    - **User DB**: Handles user sessions, roles, and credentials
    - **Item DB**: Stores vendor items
    - **Order DB**: Stores orders, group orders, and payment data
    - **Community DB**: Handles group order communities



Level 1 DFD – Main System Modules

1. **Authentication & Role Management**
    - **Inputs**: Email/password from users (Buyer, Vendor, Admin)
    - **Outputs**: Auth success/failure, Dashboard redirection
    - **Interacts with**: User DB
2. **Item Management (Vendor)**
    - **Inputs**: Item name, description, price, stock status
    - **Outputs**: Item saved/updated/deleted confirmation
    - **Stores in**: Item DB
3. **Order Management (Customer)**
    - **Submodules**:
        - **Self Order**: Add to cart → Place order → Track → Pay
        - **Group Order**: Join group → Add items → Split bill → Place → Track
        - **Quick Order**: Single-click purchase → Pay → Fast track
    - **Outputs**: Order confirmation, tracking info
    - **Interacts with**: Order DB, Community DB
4. **Admin Oversight**
    - **Inputs**: Admin login, moderation actions
    - **Outputs**: User/item/order stats, management actions
    - **Interacts with**: All DBs for monitoring

4.3 Module Design and Organization

The Community Marketplace platform is structured into clear, purpose-driven modules to ensure maintainability, scalability, and ease of development. Each module manages a distinct part of the system, enabling smooth collaboration between vendors, community users (buyers), and admins.

---

1. Authentication & User Role Module

Purpose:
Handles login and registration for Community Users, Vendors, and Admins while assigning appropriate role-based access.

Features:

- Secure registration and login for all user types
- Session-based authentication using Express middleware
- Role redirection to the correct dashboard (community, vendor, or admin)
- Middleware integration for route protection

---

2. Community User Module (Buyer Dashboard)

Purpose:
Enables community members to interact with the marketplace through browsing, ordering, and managing group purchases.

Features:

- Browse items from vendors (approved listings only)
- Add items to cart and place orders
- Join or initiate group orders within a community
- View order history
- Manage user profile and logout

---

3. Vendor Module (Vendor Dashboard)

Purpose:
Allows vendors to manage their products, monitor sales, and update information.

Features:

- Add, edit, or delete items
- Track item listings and stock
- View analytics on orders and performance
- Update vendor profile and settings
- View incoming orders from community members

---

4. Admin Module

Purpose:
Provides centralized control for platform oversight, user management, and community coordination.

Features:

- Manage all communities and users (vendors and buyers)
- Monitor platform-wide order activity
- Oversee group orders and validate data consistency
- Perform moderation and ensure policy compliance

---

5. Order Management Module

Purpose:
Processes and manages all order types, ensuring seamless handling and tracking.

Features:

- **Self Orders:** Standard cart-to-order flow
- **Group Orders:** Collaborative orders across community users
- **Quick Orders:** Fast one-click orders with minimal steps
- Tracks order lifecycle: created → processed → completed

- Integrates with vendor and admin dashboards for visibility

---

6. Item Management Module

Purpose:
Central module for item catalog handling across vendors and buyers.

Features:

- Vendors can list, edit, or delete products
- Buyers can browse and filter through available items
- Supports categorization and search functionality
- Ensures visibility control via admin-approved status

---

7. View Templates Module (Frontend)

Purpose:
Renders user-facing pages using EJS templates tailored to each role.

Features:

- Dynamic EJS templates for login, signup, dashboards, and features
- Role-specific views: community, vendor, and admin
- Modular partials: headers, footers, navbars, and dashboards
- Layout structure for clean and responsive UI

---

8. Database Model Module (MongoDB)

Purpose:
Manages the schema for all persistent data in the platform.

Features:
a) CommunityUser Model:

- Stores buyer info: name, email, password, community affiliation

b) VendorUser Model:

- Stores vendor details including business data and credentials

c) VendorItem Model:

- Captures item title, description, price, stock, and vendor reference

d) Cart Model:

- Tracks items added by community users prior to checkout

e) Order Model:

- Stores order details, statuses, and payment information

f) GroupOrder Model:

- Facilitates shared orders, split payments, and coordination

g) Community Model:

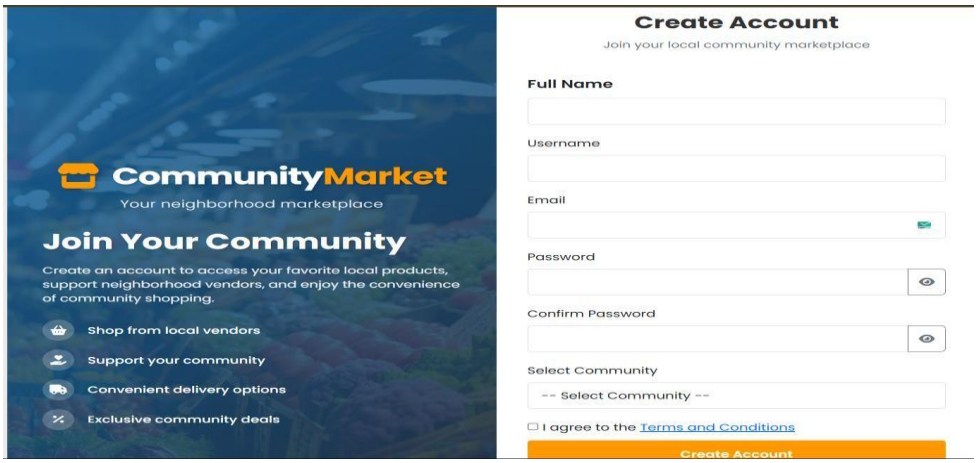
- Manages group definitions, members, and shared configurations

CHAPTER 5

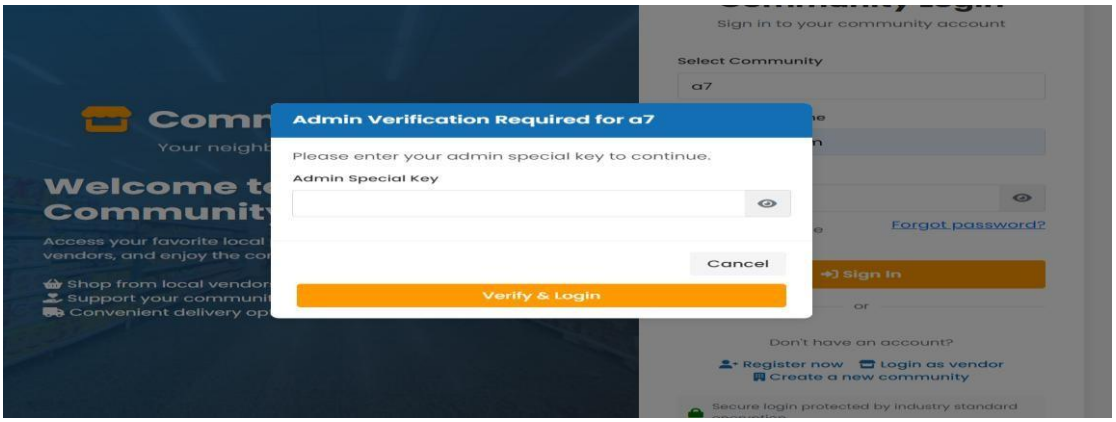
OUTPUT SCREENSHOTS



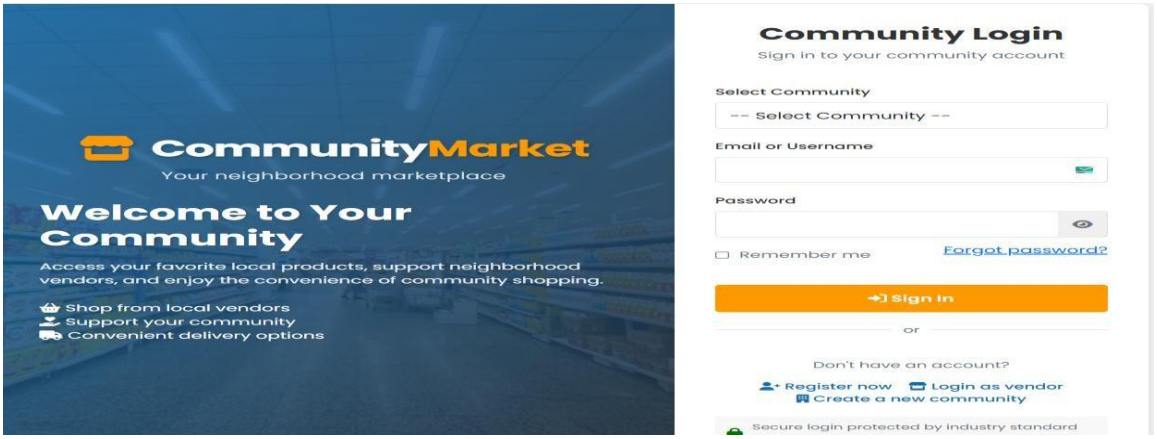
Screenshot 5.1: Home page



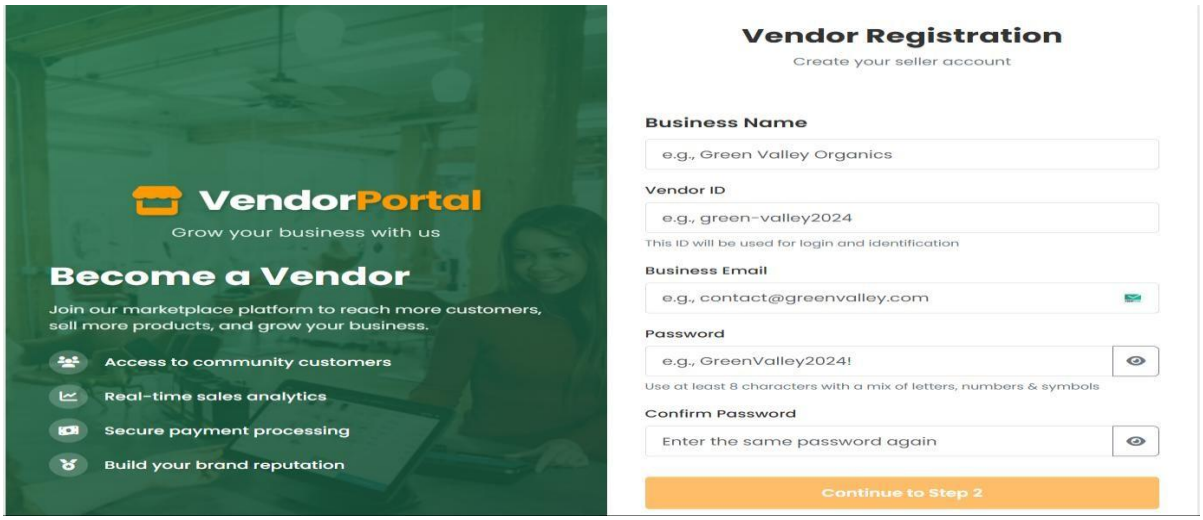
Screenshot 5.2: Community User Sign-Up page



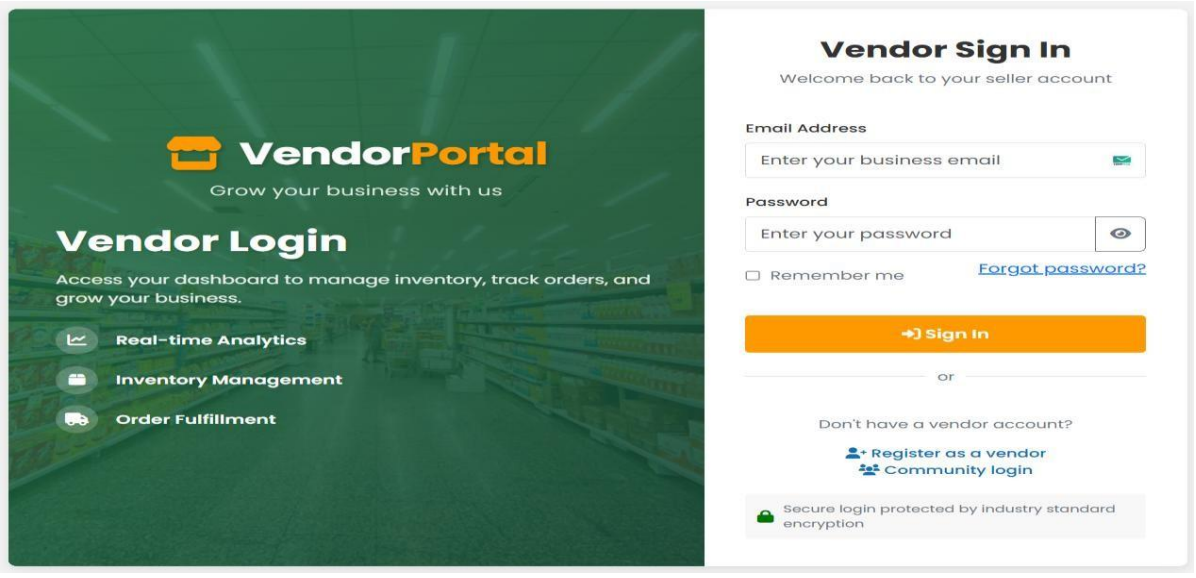
Screenshot 5.3: Community Admin Login page



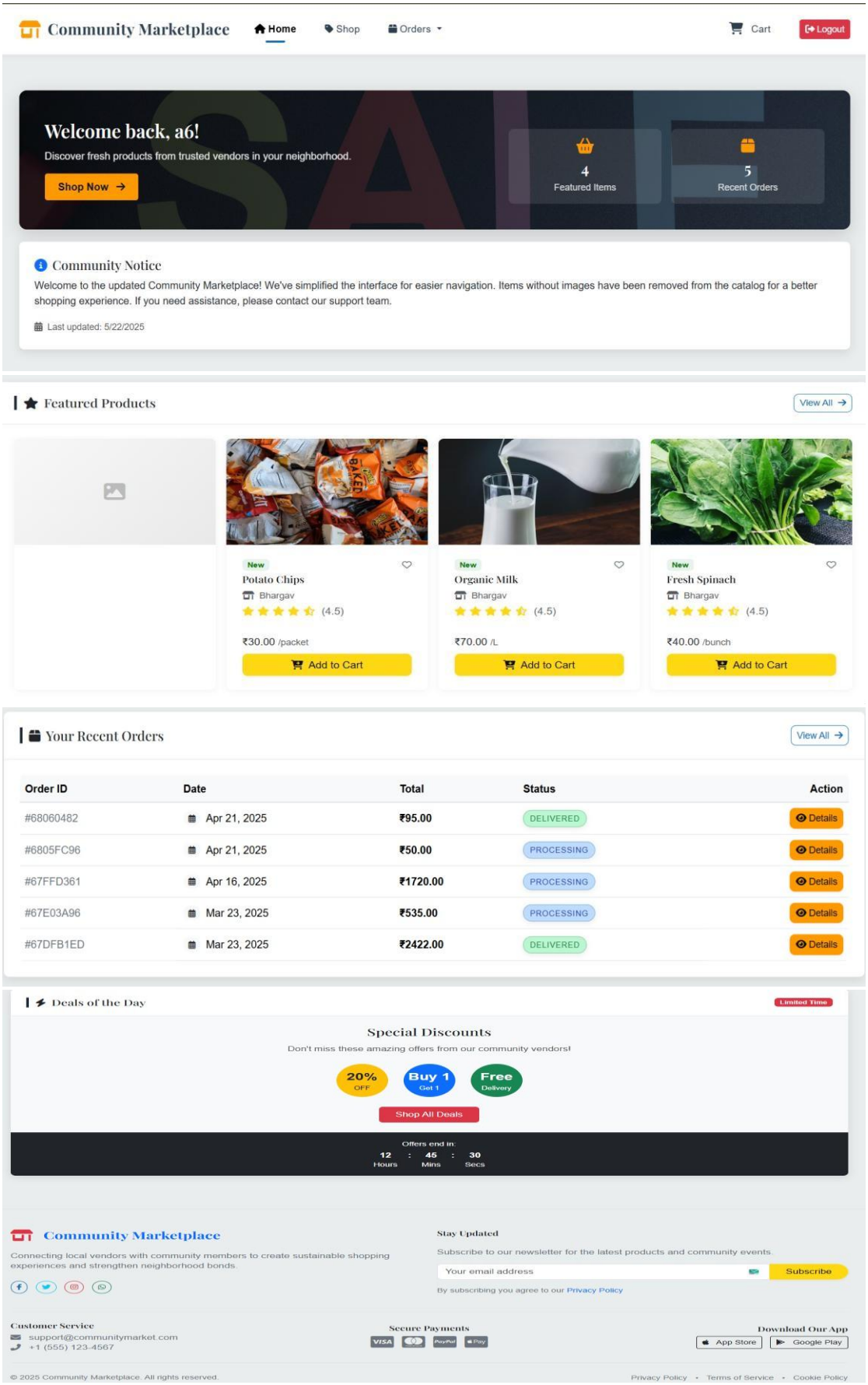
Screenshot 5.4: Community User Login page



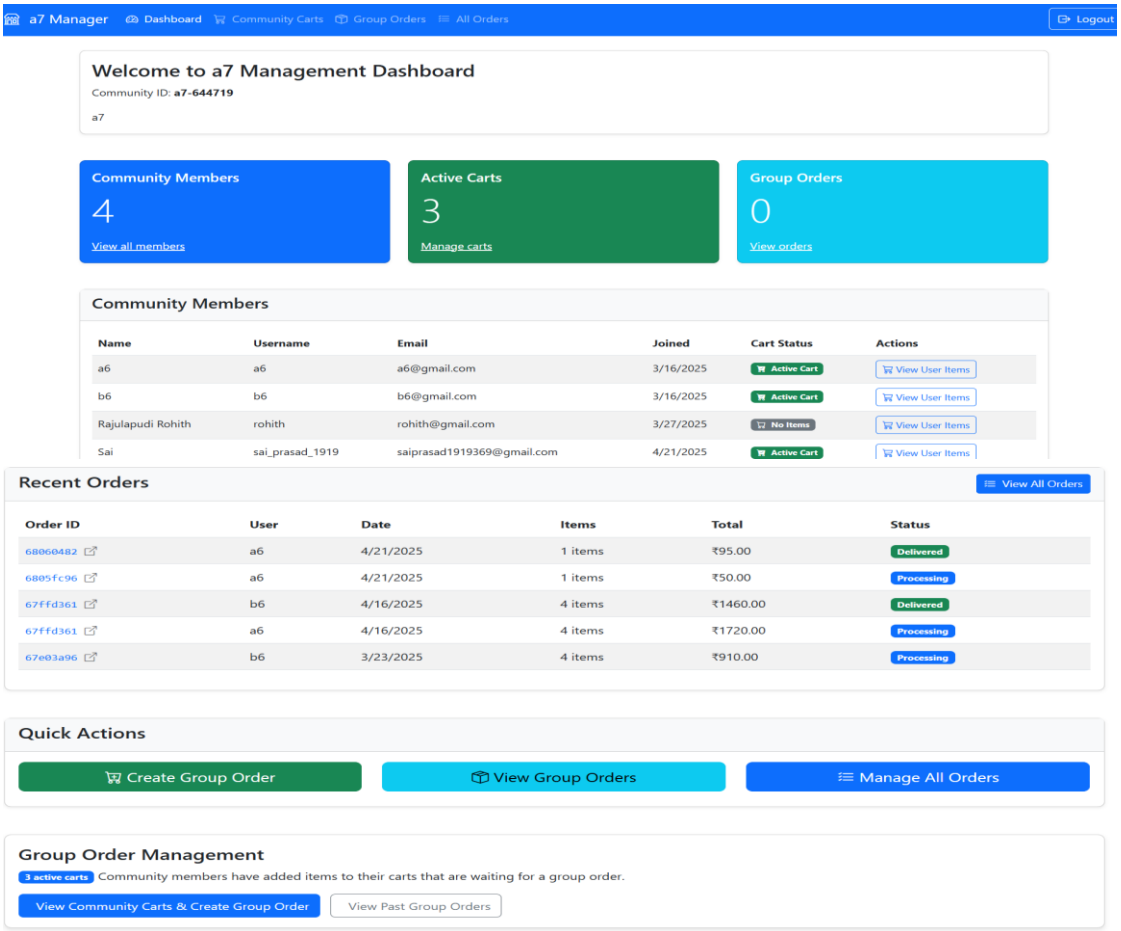
Screenshot 5.5: Vendor Sign-Up page



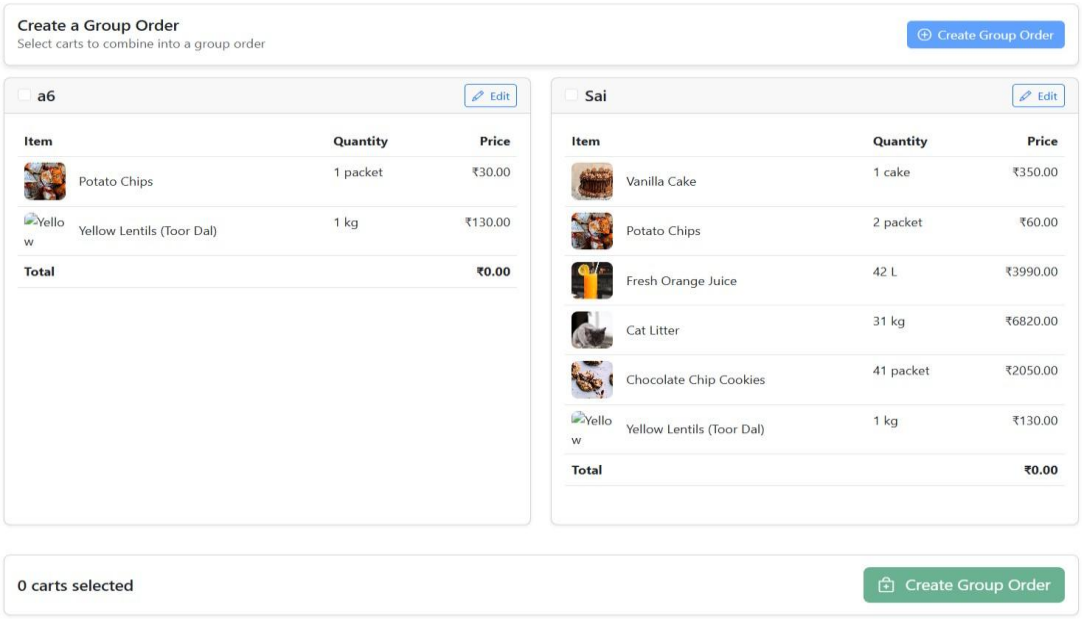
Screenshot 5.6: Vendor Login page



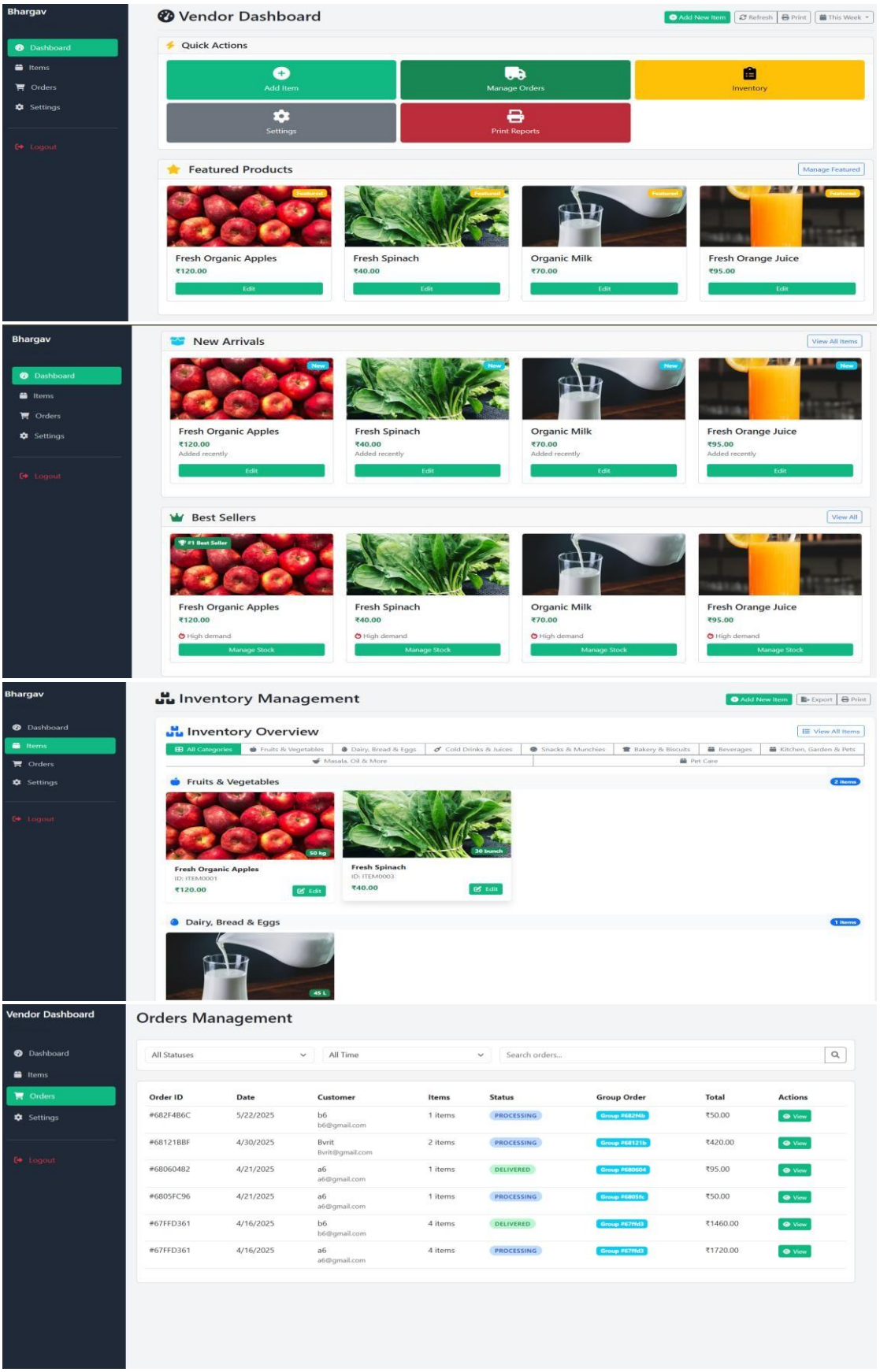
Screenshot 5.7: Community User Dashboard



Screenshot 5.8: Community Admin Dashboard



5.9: Community Order Page



Screenshot 5.10: Vendor Dashboard

Description:

The homepage of the vendor dashboard showcases two primary sections: “New Arrivals” and “Best Sellers.” These sections visually represent the latest added items and most in-demand products respectively. Items are displayed with essential information such as name, price, and availability status. Each product has an “Edit” or “Manage Stock” button based on context, ensuring vendors can easily update or manage inventory.

Newly added products are flagged with a “New” badge for better visibility, while popular items display a “#1 Best Seller” ribbon along with a “High Demand” indicator. This helps customers quickly identify trusted and trending products.

From the vendor's perspective, the dashboard allows complete control over product visibility and stock levels. Vendors can edit prices, descriptions, and images in real-time. The UI is designed for clarity and ease of use, even for users unfamiliar with digital systems. The sidebar menu includes quick access options such as Dashboard, Items, Orders, and Settings. A secure Logout function ensures session safety. Each page maintains a clean, responsive design, ideal for both mobile and desktop environments.

The Orders Management panel is another critical component. It provides a comprehensive view of all incoming orders with filters by status and time. Each order entry includes Order ID, Date, Customer Info, Item Count, Status, Group Order Tag, Total Cost, and a “View” button.

Order statuses such as “Processing” and “Delivered” are color-coded for clarity. Group Order IDs help in tracking bulk or collective purchases, useful in community or neighborhood buying groups. This dashboard structure ensures vendors do not miss important sales opportunities or customer communication. The integrated search function further improves usability for locating specific orders quickly.

All platform users—vendors, buyers, and admins—interact through role-based authentication. This ensures that each user sees only the features and data relevant to their role, maintaining privacy and focus.

Admins oversee user registration, approve vendor applications, and moderate listed products to maintain quality and relevance. Any flagged items or user-reported issues can be addressed promptly by the admin panel.

The platform supports features such as Google Maps integration (for product origin or delivery zones), personalized dashboards, favorites/bookmark systems, and mobile responsiveness.
Security is emphasized through session management, role-based access control, and cloud-based backups. HTTPS and SSL certificates protect all data transfers and interactions.
Technologically, the backend is built using Node.js with Express.js for routing and session handling. MongoDB is used as the NoSQL database to store all product, user, and order information. The frontend uses HTML, CSS, JavaScript, and EJS for templating dynamic content.

Deployment is handled via Render, providing CI/CD integration through GitHub. The cloud platform supports auto-deployment, scalable hosting, and background job execution if needed for analytics or reports.

In future updates, features like integrated payment gateways, advanced analytics dashboards for vendors, buyer feedback systems, and order delivery tracking will be added. This will convert the MVP into a more robust community commerce solution.
Overall, the Community Marketplace offers a solid base for local commerce digitization. It empowers vendors with tools to manage their business efficiently while offering customers a safe, intuitive, and transparent buying experience.

The project meets academic, economic, and operational feasibility standards. It can be expanded easily and is tailored for small communities, college campuses, or local urban centers where digital transformation is just beginning.

Its user interface balances design simplicity with functionality, encouraging adoption by users with minimal digital literacy. The categorization of products, use of intuitive icons, and responsive layout contribute to a seamless user journey.

By leveraging modern web technologies and focusing on community-centric needs, the platform aligns with the broader goal of fostering digital inclusion and entrepreneurship at the grassroots level.

With verified vendor listings, structured user interactions, and real-time order tracking, it restores trust and transparency in community-level buying and selling. Whether for fruits, vegetables, dairy, or packaged items, users can rely on the system for safe and efficient transactions.

CHAPTER 6

SAMPLE CODE

>db.js :

```
const mongoose = require('mongoose');
require('dotenv').config();
if (process.env.NODE_ENV === 'production') {
    mongoose.set('debug', {
        slowMs: 500,
        logFn: (collectionName, methodName, ...methodArgs) => {
            if (methodName.startsWith('find') || methodName ===
'aggregate') {
                console.warn(`[SLOW QUERY]
${collectionName}.${methodName} (>500ms)`);          }}});
mongoose.set('autoIndex', false); }
const options = {
    serverSelectionTimeoutMS:    10000,
    socketTimeoutMS: 60000,
    connectTimeoutMS: 15000,
    retryWrites: true,
    maxPoolSize: 15,
    family: 4,
    autoIndex: false};
const connectDB = async () => {
    let retries = 5;
    while (retries > 0) {
        try {
            await mongoose.connect(process.env.MONGO_URI, options);
            console.log('MongoDB successfully connected');
            return;
        } catch (error) {
            console.error('MongoDB connection error:', error.message);
            if (error.name === 'MongoServerSelectionError' && retries > 0){
                console.log(`Connection attempt failed. Retrying... (${retries}
attempts left)`);
                retries--;
                await new Promise(resolve => setTimeout(resolve, 2000));
            } else {
```

```
            console.error('Failed to connect to MongoDB after multiple
attempts');
            process.exit(1);}}}
 console.error('Could not connect to MongoDB. Check your connection
string and ensure MongoDB is running.');
   process.exit(1); };
module.exports = connectDB;
```

>communityLogin.js :

```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-
scale=1.0">
   <title>Community Login - Community Marketplace</title>
   <%- include('partials/favicon') %>
   <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstr
ap.min.css" rel="stylesheet">
   <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.0.0/css/all.min.css">
   <link rel="preconnect" href="https://fonts.googleapis.com">
   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
   <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;4
00;500;600;700&display=swap" rel="stylesheet">
   <style>
      :root {
         --primary-color: #ff9900; /* Amazon-inspired orange */
         --secondary-color: #146eb4; /* Amazon-inspired blue */
         --accent-color: #232f3e; /* Amazon-inspired dark blue/black
*/
         --light-bg: #f8f9fa;
         --dark-text: #333;
         --light-text: #6c757d;
         --success: #007600; /* Amazon-inspired green */
         --border-color: #ddd;
         --hover-color: #febd69; /* Amazon-inspired hover orange */
      }
```

```
    .login-banner  {
            padding: 30px;
            text-align: center;
        }
      }
   </style>
</head>

<body>
    <div class="login-container">
        <div class="login-banner">
            <div class="banner-content">
                <div class="logo-area">
                    <div class="logo">
                        <i class="fas fa-store"></i>
Community<span>Market</span>
                    </div>
                    <div class="tagline">Your neighborhood
marketplace</div>
                </div>
                <h1>Welcome to Your Community</h1>
                <p>Access your favorite local products, support
neighborhood vendors, and enjoy the convenience of community
shopping.</p>
                <div class="community-features">
                    <div class="feature">
                        <i class="fas fa-shopping-basket"></i>
                        <span>Shop from local vendors</span>
                    </div>
                    <div class="feature">
                        <i class="fas fa-hand-holding-heart"></i>
                        <span>Support your community</span>
                    </div>
                    <div class="feature">
                        <i class="fas fa-truck"></i>
                        <span>Convenient delivery options</span>
                    </div>
                </div>
            </div>
        </div>
        <div class="login-form">
            <div class="login-header">
```

```
            <h2>Community Login</h2>
            <p>Sign in to your community account</p>
        </div>

        <%- include('partials/messages') %>

        <form action="/community/login" method="POST"
id="communityLoginForm" class="needs-validation" novalidate>
            <div class="form-group">
                <label for="communityId" class="form-label">Select
Community</label>
                <select id="communityId" name="communityId"
class="form-control" required>
                    <option value="">-- Select Community --</option>
                    <% if (communities && communities.length > 0) { %>
                        <% communities.forEach(function(community) { %>
                            <option value="<%= community._id %>"><%=
community.name %></option>
                        <% }); %>
                    <% } %>
                </select>
                <div class="invalid-feedback">Please select a
community.</div>
            </div>
            <div class="form-group">
                <label for="email" class="form-label">Email or
Username</label>
                <input type="text" id="email" name="email" class="form-
control" required>
                <div class="invalid-feedback">Please enter your email or
username.</div>
            </div>
            <div class="form-group">
                <label for="password" class="form-
label">Password</label>
                <div class="input-group">
                    <input type="password" id="password"
name="password" class="form-control" required>
                    <button class="btn btn-outline-secondary"
type="button" id="togglePassword">
                        <i class="fas fa-eye"></i>
                    </button>
                </div>
```

```
                <div class="invalid-feedback">Please enter your
password.</div>
            </div>

            <div class="login-options">
                <div class="remember-me">
                    <input type="checkbox" id="remember"
name="remember">
                    <label for="remember">Remember me</label>
                </div>
                <a href="#" class="forgot-password">Forgot
password?</a>
            </div>

            <button type="submit" class="btn btn-primary">
                <i class="fas fa-sign-in-alt"></i> Sign In
            </button>
        </form>

        <div class="divider">or</div>

        <div class="login-footer">
            <p>Don't have an account?</p>
            <a href="/community/register"><i class="fas fa-user-
plus"></i> Register now</a>
            <a href="/vendor/login"><i class="fas fa-store"></i> Login
as vendor</a>
            <a href="/community/create"><i class="fas fa-
building"></i> Create a new community</a>
        </div>

        <div class="security-notice">
            <i class="fas fa-lock"></i>
            <span>Secure login protected by industry standard
encryption</span>
        </div>
    </div>
</div>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap
.bundle.min.js"></script>
<script>
```

```
        // Wait for document to be fully loaded
        document.addEventListener('DOMContentLoaded', function () {
            // Get references to important elements
            const loginForm =
document.querySelector('form[action="/community/login"]');
            const adminKeyModal = new
bootstrap.Modal(document.getElementById('adminKeyModal'));
            let savedFormData = null;
            let communityName = '';

            // Setup password toggle functionality
            const setupPasswordToggle = (buttonId, inputId) => {
                const toggleBtn = document.getElementById(buttonId);
                if (toggleBtn) {
                    toggleBtn.addEventListener('click', function() {
                        const input = document.getElementById(inputId);
                        if (input) {
                            if (input.type === 'password') {
                                input.type = 'text';
                                this.innerHTML = '<i class="fas fa-eye-
slash"></i>';
                            } else {
                                input.type = 'password';
                                this.innerHTML = '<i class="fas fa-eye"></i>';
                            }
                        }
                    });
                }
            };

            // Setup both password toggles
            setupPasswordToggle('togglePassword', 'password');
            setupPasswordToggle('toggleAdminKeyModal',
'adminKeyInput');

            // Add whitespace validation to inputs
            const addWhitespaceValidation = (input) => {
                if (!input) return;

                input.addEventListener('input', function() {
                    if (this.value.startsWith(' ')) {
                        this.setCustomValidity('Input cannot start with
spaces');
```

```
                    this.classList.add('is-invalid');
                } else {
                    this.setCustomValidity('');
                    if (this.classList.contains('is-invalid') &&
this.checkValidity()) {
                        this.classList.remove('is-invalid');
                    }
                }
            });

            // Initial check
            if (input.value && input.value.startsWith(' ')) {
                input.setCustomValidity('Input cannot start with
spaces');
                input.classList.add('is-invalid');
            }
        };

        // Apply whitespace validation to all text inputs
        const textInputs =
loginForm.querySelectorAll('input[type="text"], input[type="password"],
input[type="email"], textarea');
        textInputs.forEach(addWhitespaceValidation);

        // Handle form submission with admin check
        loginForm.addEventListener('submit', async function (e) {
            e.preventDefault();

            // Validate form (Bootstrap validation)
            if (!loginForm.checkValidity()) {
                e.stopPropagation();
                loginForm.classList.add('was-validated');
                return;
            }

            // Check for whitespace errors
            let hasWhitespaceError = false;
            textInputs.forEach(input => {
                if (input.value && input.value.startsWith(' ')) {
                    input.setCustomValidity('Input cannot start with
spaces');
                    input.classList.add('is-invalid');
                    hasWhitespaceError = true;
```

```
            }
        });

        if (hasWhitespaceError) {
            loginForm.classList.add('was-validated');
            return;
        }

        // Prepare form data
        const formData = new FormData(loginForm);

        try {
            // Check if user is admin
            const response = await fetch('/community/check-admin',
{
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                },
                body: JSON.stringify(Object.fromEntries(formData))
            });

            const data = await response.json();

            if (data.isAdmin) {
                // Save form data and show admin key modal
                savedFormData = formData;
                communityName = data.communityName || '';

                // Update modal title with community name if available
                const modalTitle =
document.getElementById('adminKeyModalLabel');
                if (modalTitle && communityName) {
                    modalTitle.textContent = `Admin Verification
Required for ${communityName}`;
                }

                adminKeyModal.show();
            } else {
                // Regular user - submit form normally
                loginForm.submit();
            }
        } catch (error) {
```

```
                console.error('Error:', error);
                loginForm.submit();

            // Handle admin key submission
            document.getElementById('submitAdminKey').addEventListen
er('click', function () {
                const adminKey =
document.getElementById('adminKeyInput').value;
                if (!adminKey) return;

                if (savedFormData) {
                    savedFormData.append('adminKey',       adminKey);

                    // Create a new form and submit
                    const newForm = document.createElement('form');
                    newForm.method = 'POST';
                    newForm.action = '/community/login';

                    for (let [key, value] of savedFormData.entries()) {
                        const input = document.createElement('input');
                        input.type = 'hidden';
                        input.name = key;
                        input.value = value;
                        newForm.appendChild(input);
                    }

                    document.body.appendChild(newForm);
                    newForm.submit();
                }
            });

            // Pre-fill email or username if provided
            <% if (locals.email) { %>
                const emailInput =
document.querySelector('input[name="email"]');
                if (emailInput) emailInput.value = '<%= email %>';
            <% } %>

            // Automatically show admin key modal if required by server
            <% if (locals.requireAdminKey) { %>
                setTimeout(() => {
                    adminKeyModal.show();
                }, 100);
```

```
            <% } %>
        });
    </script>

    <!-- Admin Key Modal -->
    <div class="modal fade" id="adminKeyModal" data-bs-
backdrop="static" tabindex="-1"
        aria-labelledby="adminKeyModalLabel"  aria-hidden="true">
        <div  class="modal-dialog  modal-dialog-centered">
            <div  class="modal-content">
                <div  class="modal-header">
                    <h5 class="modal-title" id="adminKeyModalLabel">Admin
Verification Required</h5>
                </div>
                <div  class="modal-body">
                    <p class="text-muted mb-3">Please enter your admin
special key to continue.</p>
                     <form id="adminKeyForm">
                        <div  class="form-group">
                            <label for="adminKeyInput" class="form-
label">Admin Special Key</label>
                            <div class="input-group">
                                <input type="password" class="form-control"
id="adminKeyInput" name="adminKey" required>
                                <button class="btn btn-outline-secondary"
type="button"  id="toggleAdminKeyModal">
                                    <i class="fas fa-eye"></i>
                                </button>
                            </div>
                        </div>
                    </form>
                </div>
                <div  class="modal-footer">
                    <button type="button" class="btn btn-secondary" data-
bs-dismiss="modal">Cancel</button>
                    <button type="button" class="btn btn-primary"
id="submitAdminKey">Verify & Login</button>
                </div>
            </div>
        </div>
    </div>
</body>
</html>
```

CHAPTER 7

CONCLUSION

As we reflect on the journey of building the **Community Marketplace**, what stands out most is not just the code we wrote or the modules we implemented—it's the people we built this for. The community member excited to place their first group order, the vendor finally able to reach more local customers, the admin working behind the scenes to keep everything running smoothly—these are the real stories that give meaning to our work.

This platform is more than a marketplace. It has become a **shared space**—a digital community center where trust is built, support is shared, and local economies thrive. Every cart filled, every group order completed, and every product delivered is a reminder that we are helping people connect, contribute, and grow together.

Yes, the data matters—efficient order processing, active vendors, smooth admin oversight—but the real success lies in the **moments of impact**: A vendor expanding their reach, a community member feeling empowered, or a local group collaborating on a bulk order to save costs and support their neighborhood.

The **Community Marketplace** proves that technology is most powerful when it empowers real people. As we look to the future, we remain committed to refining the experience, enhancing trust, and staying rooted in the values of community, transparency, and service.

In a world moving fast, this project is a reminder that **progress doesn't just mean speed—it means connection**. And with this platform in their hands, our users can continue to write stories of shared success, one transaction at a time.

CHAPTER 8

FUTURE WORK

As our **Community Marketplace** continues to evolve, we remain focused on one core value: **building trust and ease within local communities**. Every product listed, every message sent, and every transaction completed reflects real people connecting over real needs. Our vision is to expand and improve the platform with features that make buying, selling, and interacting safer, smarter, and more community-driven.

1. **In-App Messaging – Safer, Direct Communication:** To keep buyers and sellers connected within a secure environment, an in-app chat system will allow users to negotiate, ask questions, and share updates—without relying on external platforms.
2. **Delivery and Pickup Options – Flexible Exchange Models:** We plan to introduce local delivery and scheduled pickup options, giving users more flexibility in how they complete transactions, especially for bulky or urgent items.
3. **Ratings & Verified Reviews – Build Trust with Every Transaction:** Buyers and sellers will be able to leave reviews for each other after deals are completed. This builds a transparent, trustworthy community and helps users make better decisions based on real feedback.
4. **Smart Listing Suggestions – Personalized Browsing Experience:** An AI-powered engine will soon recommend products based on user behavior, location, interests, and recent searches—making it easier for users to discover items they actually want.
5. **Community Events & Services Section – Beyond Just Products:** The marketplace will expand to include local services and event listings. From garage sales and tutoring offers to repair services, this will help users engage with their community more fully.
6. **Verified Seller Badges – Confidence in Who You Buy From:** Trusted and frequently active sellers will be eligible for verification badges. This adds credibility to listings and increases buyer confidence in the platform.
7. **Dispute Resolution Center – Safer Transactions, Happier Users:** To handle rare cases of misunderstanding or fraud, a simple dispute resolution process will help both parties resolve issues fairly, enhancing the overall reliability of the marketplace.

CHAPTER 9

References

[1] Kumar, R., & Patel, D. (2022). “Designing Role-Based Dashboards for Multi-User Marketplaces,” *International Journal of Web Engineering*, vol. 18(3), pp. 112–125.

[2] Roy, S., & Mehra, K. (2021). “A Secure Node.js Architecture for E-Commerce Platforms: A Case Study,” *Journal of Computing and Digital Systems*, vol. 15(2), pp. 55–68.

[3] Jain, A., & Verma, P. (2020). “Community-Driven Marketplace Design Using MongoDB and Express.js,” *IJERT*, vol. 9(7), pp. 72–78.

[4] D’Souza, T., & Reddy, L. (2023). “User-Centric Interface Models for Online Retail Applications,” *HCI Proceedings of the 27th International Conference on Software Engineering*, pp. 210–220. Springer.

[5] Mishra, A., & Thomas, V. (2022). “Enhancing Local Commerce via Digital Marketplaces: Case Studies from Rural India,” *ACM Digital Markets Journal*, vol. 11(1), pp. 35–48.